BST vs AVL Tree — Short Report

1. Differences Observed

1.1 Tree Height

Insertion Order	BST Height	AVL Height
Random (25 books)	~6–8	~5
Sorted (25 books)	25	5

When books were inserted in **random order**, both trees produced comparable heights. The BST was slightly taller but still reasonably balanced by chance.

When the same books were inserted in **ascending sorted order**, the BST degenerated into a linked list of height 25 — every new node became the right child of the previous one. The AVL tree, through automatic rotations, maintained a height of approximately $\log_2(25) \approx 5$ regardless of insertion order.

1.2 Search Performance

Insertion Order	BST Steps (ID 930)	AVL Steps (ID 930)
Random	~5–7	~5
Sorted	25	5

Search in the AVL tree always completed in **O(log n)** steps because the tree is guaranteed to be balanced after every insert and delete. The BST matched AVL performance on random data but degraded to **O(n)** — a full linear scan — on sorted data, visiting every single node.

1.3 Insertion and Deletion Overhead

The **BST** performs straightforward insertions and deletions with no rebalancing cost — each operation simply places the node and returns.

The **AVL tree** performs additional work after every insertion and deletion:

- Updates the height of each node along the insertion/deletion path
- Computes the balance factor at each ancestor
- Performs rotations (single or double) when the balance factor exceeds ±1

This means AVL insertions and deletions are slightly slower per operation due to the rotation and height-maintenance overhead, but the guarantee of a balanced tree pays off during search-heavy workloads.

1.4 Self-Balancing Mechanism

Aspect	BST	AVL
Balancing	None	Automatic via rotations
Rotation types	N/A	LL (right), RR (left), LR, RL
Height guarantee	O(n) worst, O(log n) average	O(log n) always
Code complexity	Simple	Moderate (rotation logic)

---

2. When to Use BST vs AVL

✅ Use a Regular BST when:

- **Data is inserted in random order.** A BST built from random inputs is probabilistically balanced and performs nearly as well as an AVL tree without the overhead.
- **The dataset is small.** For a few dozen or hundred elements, the height difference between a BST and AVL is negligible, and the simpler code is an advantage.
- **Insertions and deletions are very frequent, and searches are rare.** If you mostly build the tree and rarely query it, the rebalancing cost of AVL may not be justified.

- **Memory and simplicity are priorities.** BST nodes don't need a height field, and the code is shorter and easier to maintain.

✅ Use an AVL Tree when:

- **Data may arrive in sorted or nearly-sorted order.** This is common in real-world systems — book IDs, timestamps, and auto-increment keys are often inserted sequentially. AVL prevents the degenerate linked-list case entirely.
- **Search is the dominant operation.** In a library system, users search for books far more often than new books are added or removed. The O(log n) search guarantee of AVL directly improves user-facing responsiveness.
- **Predictable performance is required.** Applications with latency constraints (e.g., real-time systems) cannot tolerate the O(n) worst-case of an unbalanced BST. AVL guarantees O(log n) for all operations.
- **The tree is long-lived and undergoes many deletions.** Repeated deletions in a BST can gradually unbalance it. AVL trees re-balance after every deletion, maintaining performance over the tree's lifetime.